

DNS + Anycast Hybrid Routing

Proof-of-Concept Documentation

QUIC Server-Side Migration for Anycast Connection Stability

PoC Implementation Guide • July 2026

This document describes a proof-of-concept implementation that combines DNS-based GeoDNS resolution with simulated anycast routing and QUIC server-side migration. The PoC demonstrates how QUIC's preferred_address mechanism (RFC 9000, Section 9.6) can solve the fundamental instability problem of anycast-routed connections: when BGP route changes redirect traffic to a different Point of Presence (PoP), existing connections break. By migrating connections from the anycast PoP to a unicast backend server immediately after the TLS handshake, the connection becomes immune to subsequent anycast route changes. This two-phase routing approach combines the fast initial routing of anycast with the stability of unicast for connection lifetime.

1. Anycast in the Real World

What Is Anycast?

Anycast is a network addressing and routing methodology where a single IP address is announced from multiple geographic locations simultaneously via BGP (Border Gateway Protocol). When a client sends a packet to an anycast IP, the Internet's routing infrastructure delivers it to the topologically nearest instance -- the one with the shortest AS-path or lowest IGP metric. Major CDNs (Cloudflare, Akamai, Fastly) and DNS providers (Google Public DNS 8.8.8.8, Cloudflare 1.1.1.1) rely on anycast to distribute traffic across dozens or hundreds of Points of Presence (PoPs) worldwide.

- **Multiple PoPs, one IP:** The same IP prefix (e.g., 1.1.1.0/24) is announced from data centers in New York, London, Tokyo, etc.
- **BGP does the routing:** Internet routers select the shortest AS-path, effectively directing each client to the geographically nearest PoP
- **No client configuration:** Clients see a single IP address and are unaware that multiple servers share it
- **Ideal for stateless protocols:** DNS (UDP, single request-response) works perfectly with anycast since each query is independent

Why Anycast Breaks Stateful Connections

While anycast excels at routing individual packets to the nearest server, it is fundamentally incompatible with stateful protocols like TCP and QUIC. The core issue is that BGP routing decisions can change at any time -- due to link failures, congestion-based rerouting, planned maintenance, or even normal BGP path exploration. When a route changes, subsequent packets for an existing connection may be delivered to a different PoP that has no state for that connection.

- **TCP:** The new PoP receives a packet with an unknown 5-tuple. It sends RST, and the connection dies immediately. Retransmission timers on the client side eventually expire (30-120 seconds), but the connection is already doomed.
- **QUIC:** QUIC uses Connection IDs instead of 5-tuples, which helps with NAT rebinding and client migration, but does NOT help with anycast flaps: the new PoP still has no TLS state for the connection and cannot decrypt the packets.
- **Cross-PoP state sharing:** Synchronizing connection state across globally distributed PoPs in real-time is impractical at scale. The state includes TLS session keys, sequence numbers, flow control windows, and application state -- hundreds of bytes that change with every packet.

Anycast is fundamentally incompatible with stateful connections: BGP route changes silently redirect packets to PoPs that cannot process them.

The Solution: Two-Phase Routing with QUIC Migration

QUIC server-side migration introduces a two-phase routing model that combines the strengths of anycast (fast initial routing) with the stability of unicast (reliable connection lifetime). The insight is simple: use anycast only for the initial handshake (Phase 1), then immediately migrate the connection to a dedicated unicast backend (Phase 2). Once migrated, the connection is on a direct client-to-backend path that is completely independent of anycast routing.

- **Phase 1 -- Anycast Discovery (1 RTT):** Client connects to the anycast VIP. BGP routes the Initial packet to the nearest PoP. The PoP completes the TLS 1.3 handshake and includes a preferred_address

transport parameter advertising the unicast backend address.

- **Phase 2 -- Unicast Stability (connection lifetime):** Client validates the new path via PATH_CHALLENGE/PATH_RESPONSE and migrates. All subsequent traffic flows on the direct unicast path. Anycast routing changes have zero effect.

Two-phase routing: use anycast for fast discovery (1 RTT), then pin to unicast for the entire connection lifetime. Best of both worlds.

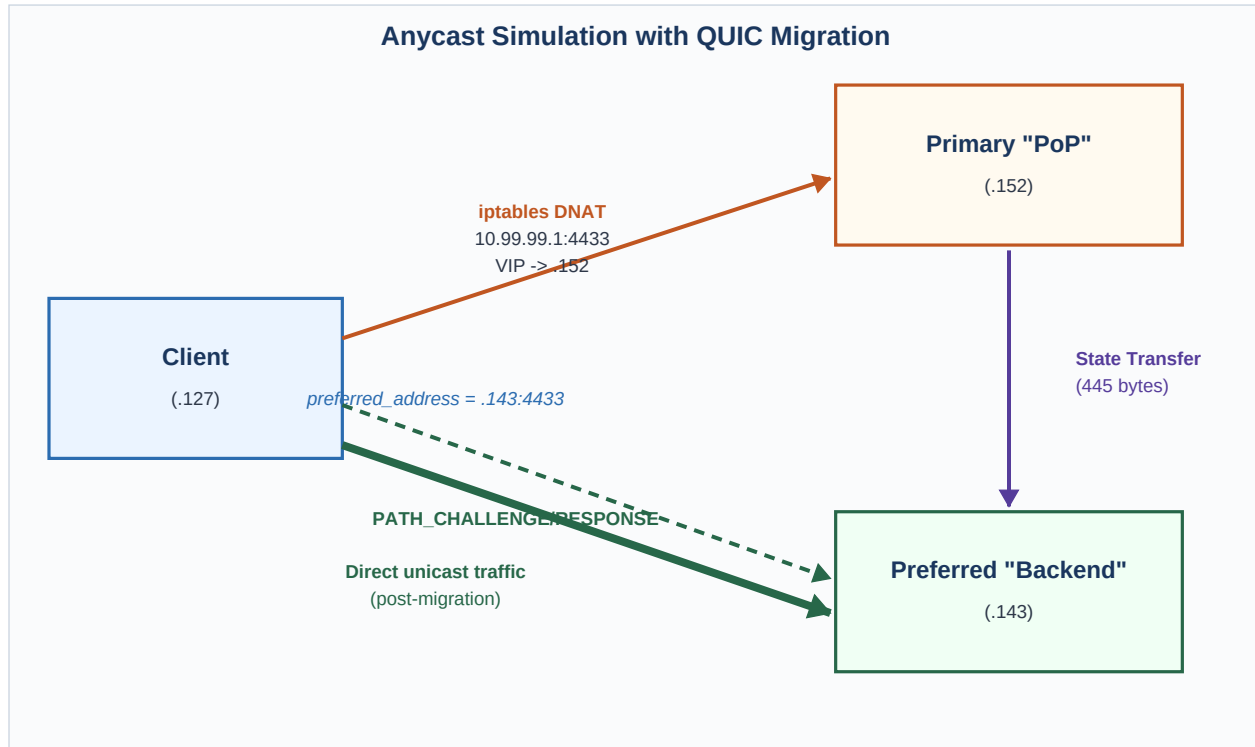
2. Architecture

Testbed Layout

The PoC uses our existing four-machine LAN testbed (141.217.168.x subnet) with the following role assignments:

Role	Machine	IP Address	Description
VIP (Anycast)	Client loopback	10.99.99.1	Simulated anycast address (loopback + iptables DNAT)
PoP (Primary)	opti7040	141.217.168.152	Simulated nearest anycast Point of Presence
Backend (Preferred)	homeserver2	141.217.168.143	Final unicast destination after migration
Alt PoP	Proxmox VM	141.217.168.200	Destination after simulated BGP route change
Client	optiplex7010	141.217.168.127	QUIC client (neqo-client or Firefox)

Anycast Simulation Diagram



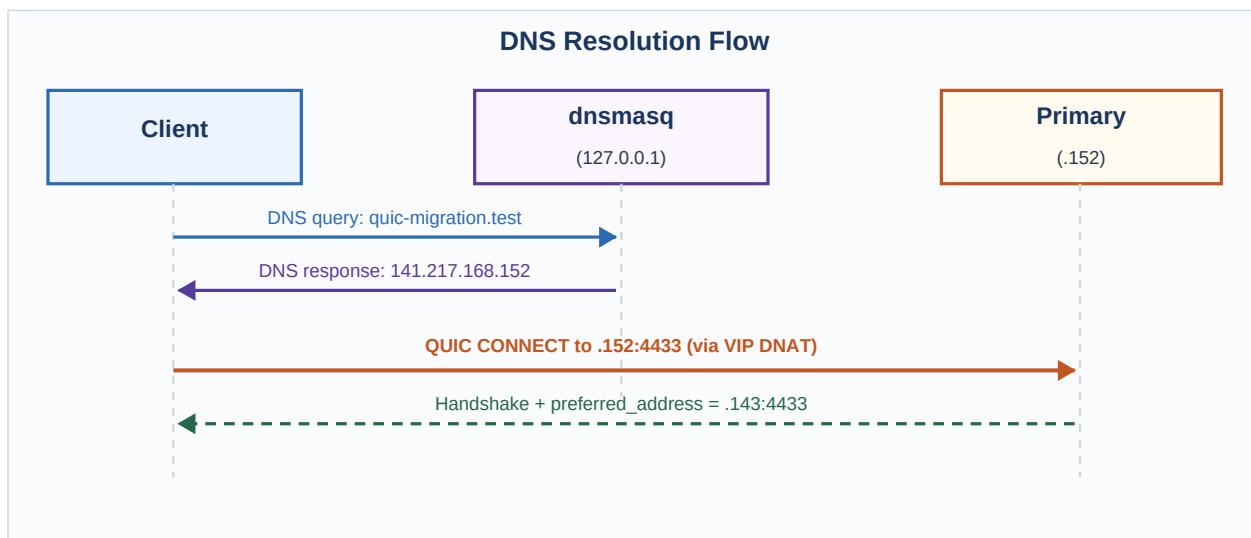
Connection Flow

The connection proceeds through three distinct phases:

- **Phase 1 (Anycast)**: Client connects to VIP 10.99.99.1:4433. iptables DNAT redirects traffic to primary server at .152 (simulates anycast nearest-PoP routing).
- **Phase 2 (Handshake + Migration)**: Primary completes QUIC handshake, advertises preferred_address = 141.217.168.143:4433. Client sends PATH_CHALLENGE to .143.
- **Phase 3 (Unicast)**: Client validates path to .143, migrates. Connection now runs directly client <-> .143 on a stable unicast path.

3. DNS Resolution Flow

A local dnsmasq instance resolves a test domain (e.g., quic-migration.test) to the primary server IP. This simulates GeoDNS behavior where a CDN's authoritative DNS returns the IP of the nearest anycast PoP. In a production deployment, this would be a real GeoDNS service (e.g., AWS Route53 Geolocation, Cloudflare Load Balancing) returning the anycast VIP.



- dnsmasq resolves the test domain to the primary IP (simulates GeoDNS)
- Client resolves domain, connects to VIP, DNAT routes to primary
- In production, the DNS response would return the anycast VIP directly; BGP routing handles PoP selection transparently

DNS TTL vs Anycast Failover vs QUIC Migration

These three mechanisms operate at fundamentally different time scales for traffic redirection:

Mechanism	Failover Time	Granularity	Connection Impact
DNS TTL expiry	30s - 24h (TTL-dependent)	Per-domain, all clients	None (only affects new connections after TTL)
BGP/Anycast convergence	1s - 90s (BGP timers)	Per-prefix, per-router	Catastrophic: existing connections break
QUIC preferred_address	~1 RTT (~1-50ms LAN/WAN)	Per-connection	None: migration is transparent to application

QUIC migration operates at per-connection granularity in ~1 RTT, making it orders of magnitude faster and more precise than DNS or BGP failover.

4. iptables DNAT Explained

VIP Setup on Loopback Interface

The virtual IP (VIP) 10.99.99.1 is assigned to the loopback interface rather than a physical NIC. This is a deliberate design choice:

- **Why loopback, not a real interface:** Assigning the VIP to a physical interface would cause ARP responses on the LAN, potentially conflicting with other hosts. The loopback interface is purely local -- it accepts traffic from the kernel's network stack without any L2 (Ethernet/ARP) involvement.

- **Kernel behavior:** When the client application connects to 10.99.99.1, the kernel finds the VIP on the loopback interface and would normally deliver the packet locally. The DNAT rule intercepts the packet in the OUTPUT chain before local delivery, rewriting the destination IP to the real primary server address.
- **Production parallel:** In real anycast, the VIP would be the anycast IP announced via BGP. No loopback trick is needed -- BGP routing naturally directs packets to the nearest PoP.

VIP Assignment

```
sudo ip addr add 10.99.99.1/32 dev lo
```

DNAT Rules Explained

The iptables rules operate in the **nat** table, **OUTPUT** chain. The OUTPUT chain is used (rather than PREROUTING) because the traffic originates from the local machine. The rule matches packets destined for the VIP on UDP port 4433 (QUIC) and rewrites the destination IP.

Setup Rule (route VIP to primary):

```
sudo iptables -t nat -A OUTPUT -d 10.99.99.1 -p udp --dport 4433 -j DNAT --to-destination 141.217.168.152:4433
```

- **-t nat:** Operate on the nat table (address translation)
- **-A OUTPUT:** Append to the OUTPUT chain (locally-originated packets)
- **-d 10.99.99.1:** Match packets destined for the VIP
- **-p udp --dport 4433:** Match QUIC traffic (UDP port 4433)
- **-j DNAT --to-destination:** Rewrite destination IP to the primary server

Route Flap Rule (simulate BGP change):

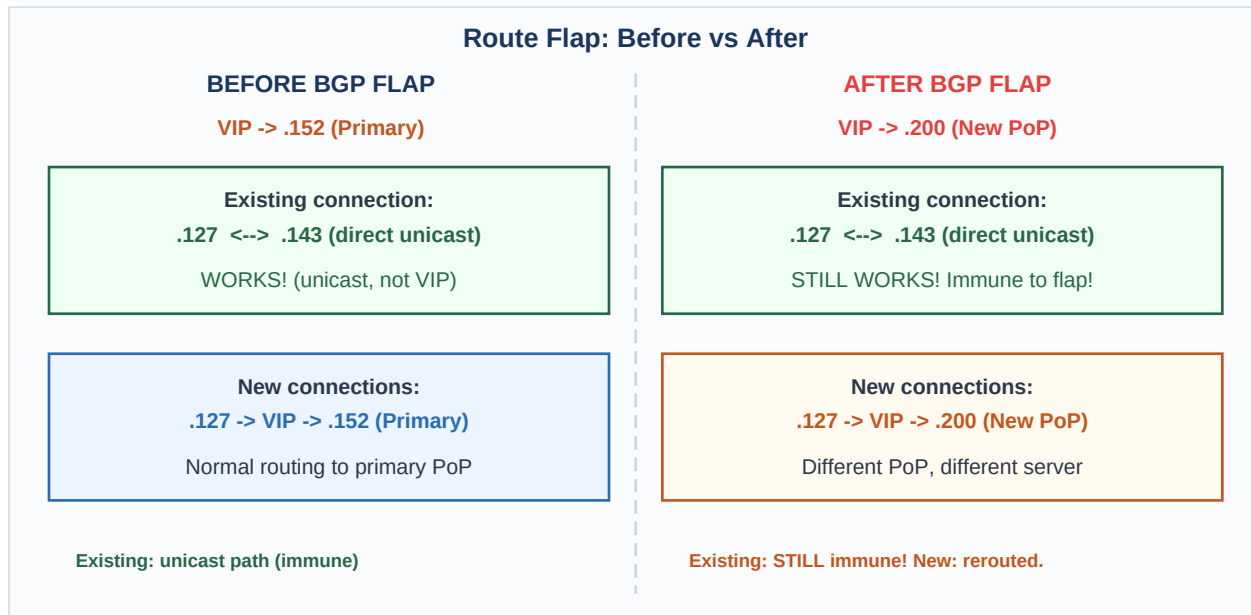
```
# Delete old rule, add new one pointing to alternate PoP
sudo iptables -t nat -D OUTPUT -d 10.99.99.1 -p udp --dport 4433 -j DNAT --to-destination 141.217.168.152:4433
sudo iptables -t nat -A OUTPUT -d 10.99.99.1 -p udp --dport 4433 -j DNAT --to-destination 141.217.168.200:4433
```

Key: After migration, the client talks directly to .143 on a unicast path. The DNAT rule only affects NEW connections through the VIP -- migrated connections bypass DNAT entirely.

5. Route Flap Resilience

The route flap test is the core demonstration of the PoC. It proves that QUIC migration makes connections resilient to anycast route changes.

Before vs After BGP Flap



Test Procedure

- **Step 1:** Establish connection through VIP -> primary (.152) -> migration -> preferred (.143). Connection is now on unicast path.
- **Step 2:** Change DNAT rule: VIP (10.99.99.1) now routes to .200 instead of .152. This simulates a BGP route change that would redirect anycast traffic to a different PoP.
- **Step 3:** Verify that the **migrated connection SURVIVES**. It is on the direct unicast path to .143 and is unaffected by the DNAT change.
- **Step 4:** Establish a **new** connection through VIP. It goes to .200 (the new PoP), demonstrating that the route change did take effect for new connections.

Result: The migrated connection survives the simulated BGP route change. New connections go to the new PoP (.200), but existing migrated connections remain stable on their unicast backend (.143).

6. Scripts

The PoC includes four shell scripts that manage the simulation environment:

Script	Purpose	Requires sudo
setup_anycast_sim.sh	Creates the VIP (10.99.99.1) on loopback, adds iptables DNAT rules to redirect VIP traffic to the primary server (.152)	Yes
setup_dnsmasq.sh	Configures a local dnsmasq instance to resolve the test domain (quic-migration.test) to the primary server IP, simulating GeoDNS	Yes

Script	Purpose	Requires sudo
test_route_flap.sh	Demonstrates connection survival during a simulated BGP route change. Changes the DNAT target from .152 to .200 while a migrated connection is active	Yes
teardown.sh	Removes all iptables rules, deletes the VIP from loopback, stops dnsmasq, and restores the original DNS configuration	Yes

7. How to Run

Step 1: Setup (on client machine)

```
sudo ./setup_anycast_sim.sh
sudo ./setup_dnsmasq.sh
```

Step 2: Start servers (on remote machines)

On **homeserver2** (preferred/backend server):

```
preferred-server 141.217.168.143:4433 9999
```

On **opti7040** (primary/PoP server):

```
primary-server 0.0.0.0:4433 141.217.168.143:4433 141.217.168.143:9999
```

Step 3: Test connection through VIP

```
neqo-client https://10.99.99.1:4433/
```

The client connects to the VIP, which is DNAT'd to the primary. The primary completes the handshake and advertises the preferred address. The client migrates to the backend, and the connection proceeds on a direct unicast path.

Step 4: Route flap test

```
sudo ./test_route_flap.sh
```

This script changes the DNAT target from .152 to .200 while a connection is active, demonstrating that migrated connections survive the route change.

Step 5: Cleanup

```
sudo ./teardown.sh
```

8. CDN Use Case: Anycast with Backend Pinning

Major CDN providers like Cloudflare, Akamai, and Fastly use anycast extensively to route clients to the nearest edge PoP. However, long-lived connections (video streaming, WebSocket, gRPC streams, large file downloads) are vulnerable to BGP flaps. QUIC migration offers a production-grade solution: **anycast for**

discovery, unicast for connection lifetime.

Cloudflare/Akamai-Style Deployment

- **Edge PoPs (anycast):** 200+ data centers worldwide announce the same IP prefix. Each PoP handles TLS handshakes and serves as the initial QUIC endpoint.
- **Origin/Mid-tier (unicast):** Dedicated backend servers with stable unicast IPs handle the actual application logic. The PoP's preferred_address points to the assigned backend for each connection.
- **Backend selection:** The PoP selects the optimal backend based on connection metadata (geo, load, content affinity) during the handshake, then migrates.
- **BGP flap immunity:** Once migrated, the connection is pinned to the unicast backend. BGP flaps at the anycast layer only affect new connections (which get routed to whichever PoP is now nearest).

Connection Lifecycle in Production

Phase	Duration	Path	Vulnerability
DNS resolution	~50ms	Client -> DNS -> anycast VIP	DNS cache poisoning
Anycast routing	~1ms (LAN) / ~20ms (WAN)	Client -> nearest PoP (BGP)	BGP flap (pre-migration)
TLS 1.3 handshake	1 RTT (~1-50ms)	Client <-> PoP	BGP flap during handshake
Migration	1 RTT (~1-50ms)	Client -> Backend (PATH_CHALLENGE)	None (validated path)
Application data	Seconds to hours	Client <-> Backend (direct unicast)	None (immune to anycast)

The vulnerability window is extremely small: only during the TLS handshake (~1 RTT) is the connection susceptible to BGP flaps. After migration completes, the connection is fully immune for its entire remaining lifetime.

State Transfer Considerations

The migration state (TLS secrets, CIDs, packet numbers) is only 445 bytes. This makes state transfer practical even at CDN scale. Our implementation supports multiple transfer backends (TCP push, Redis KV, Redis Pub/Sub, HTTP pull, file) to match different deployment architectures:

- **TCP Push (lowest latency):** Direct TCP connection from PoP to backend. Best for co-located PoP/backend pairs.
- **Redis KV (best scalability):** State stored in shared Redis. Backend polls or receives notification. Best for multi-backend deployments.
- **HTTP Pull (best security):** Backend pulls state from PoP on demand. TLS secrets never leave the PoP's memory until explicitly requested.

9. Production Deployment Considerations

From PoC to Production

The PoC uses iptables DNAT to simulate anycast on a LAN. A production deployment would replace this with real BGP anycast infrastructure:

Component	PoC (This Implementation)	Production
Anycast routing	iptables DNAT on client loopback	BGP anycast: same /24 announced from multiple PoPs
DNS resolution	Local dnsmasq (quic-migration.test)	GeoDNS (Route53, Cloudflare) or anycast DNS
PoP selection	Single primary (.152)	BGP shortest-path to nearest of N PoPs
Backend selection	Hardcoded preferred_address (.143)	Dynamic: load-based, geo-based, or content-affinity
State transfer	TCP push / Redis / HTTP (445 bytes)	Same backends, production-hardened (TLS, auth, retry)
Route simulation flap	iptables rule change	Actual BGP withdrawal / announcement
Client	neqo-client / Firefox	Any RFC 9000-compliant QUIC client

Real BGP Anycast Requirements

- **AS number:** Each PoP announces the same IP prefix from the same ASN (or via BGP communities for multi-ASN setups)
- **IP prefix:** A /24 is the minimum prefix size accepted by most transit providers. The anycast VIP sits within this prefix.
- **BGP sessions:** Each PoP peers with upstream transit providers and IXPs, announcing the anycast prefix with appropriate AS-path and communities
- **Health checks:** BGP announcements are withdrawn when a PoP is unhealthy, causing traffic to shift to the next-nearest PoP (which then migrates connections to its backend)
- **ECMP handling:** Equal-Cost Multi-Path routing may split flows across PoPs. Connection IDs in QUIC help, but pre-migration traffic must reach the same PoP.

Note: The PoC's iptables DNAT accurately models the packet-level behavior of anycast routing. The only difference is WHERE the routing decision happens (client kernel vs Internet backbone routers).

10. Comparison

The following table compares three approaches to routing clients to distributed servers, highlighting the advantages of combining QUIC migration with anycast:

Scenario	Pure Anycast	DNS-based	QUIC Migration + Anycast
Initial routing	BGP (fast, automatic)	DNS TTL (slow, cached)	BGP + migration (fast)

Scenario	Pure Anycast	DNS-based	QUIC Migration + Anycast
Route change impact	Connection breaks immediately	Wait for DNS TTL to expire	No impact on existing connections
Connection stability	Poor (any BGP flap breaks it)	Medium (stable until DNS re-resolve)	Excellent (pinned to unicast)
Data path	Through PoP always (extra hop)	Direct after DNS resolution	Direct after migration (~1 RTT)
Failover speed	BGP convergence (seconds-minutes)	DNS TTL (seconds-hours)	Instant (already on unicast)
Client complexity	None	DNS resolver logic	QUIC stack support (RFC 9000)

QUIC Migration + Anycast achieves the best of both worlds: the fast initial routing of anycast with the connection stability of direct unicast paths. The migration overhead is minimal (one additional RTT for PATH_CHALLENGE/PATH_RESPONSE), and the resulting connection is immune to all subsequent routing changes.